Portfolio Assignment for

Master of Science in Information Technology

Software Design and Programming

Estell Moore

University of Denver College of Professional Studies

11/14/2025

Faculty: Nirav Shah, M.S.

Director: Cathie Wilson, M.S.

Dean: Michael J. McGuire, MLS

Abstract

This project is an in-depth construction of a parking system that handles permit issuing and transaction charging. Throughout the ICT 4305 Denver University course, many challenges were encountered to keep things simple and scalable. Particularly, the structure of the code was constantly changing, and is outlined in this project to describe the complications and differences between each required functionality implementation. The parking system was required to handle registering customers with associated cars, cars with issued permits, charges associated with customers based on which parking lot they parked in and for how long, and ensured the system could also handle invalid entries.

CONTENTS

1. Introduction 3

2. About the Parking System 3

The Customers and Cars

The Parking Lots

3. Structuring the Project 4

4. The Experience 8

5. Conclusion 9

REFERENCES 10

APPENDICES 11

Appendix A: Test Execution, Compilation and Coverage

Appendix B: Updated Class Diagram

Appendix C: Sequence Diagram - Customer registering a Car and receiving a Parking Permit

Appendix D: Sequence Diagram - Car entering a Parking Lot, resulting in the Customer incurring a charge

Appendix E: Interaction Diagram

Introduction

This project includes a collection of classes that registers customers, issues permits, and calculates parking charges for campus parking lots. The overall idea for this is to ensure that when customers park in designated university lots, they have active and valid permits, parking lots are not full, and the customers are properly charged based on which lots they park in. The overall system had many areas of consideration when developing, and the code needed to be scalable to handle new additions and functionality expansions.

About the Parking System

The Customers and Cars

A customer can register multiple cars to their name, as well as register a single car to multiple lots. The cars are issued a permit, and are identified by their license plate number and the vehicle type to ensure when permits are being validated, it matches to the correct car. Even if the customer exists in the system, if they don't register the car they try to park, that could result in denied entry or a parking ticket. In Appendix C, a sequence diagram is developed to show the flow of a customer registering their car for a permit, whereas Appendix D is a sequence diagram developed to outline the process of the system as a customer enters a car lot and how they are expected to be charged.

The Parking Lots

In this project, two parking lots are provided: one that charges hourly, and another that charges a flat-rate with overnight fees. When a customer tries to park in a lot that they are not valid for, they are denied entry.

The North parking lot is set at a fixed rate, so when a customer enters the parking lot, they are immediately charged to enter the lot. Then, if the customer leaves their car overnight in the lot, they are charged with an overnight fee. The threshold for what defines “overnight” is 2AM the following day. As long as the customer leaves before then, they will not be charged.

The South parking lot is an hourly lot where customers are charged only based on how long they’ve been parked in the lot. There are no overnight fees, so the customers are linearly charged for however long they leave their car in the lot. Although, upon entry, if the customer enters the lot and they leave in under 1 hour, they are still charged at least 1 hours-worth of parking.

Structuring the Project

Throughout the course, working on this project eventually grew into a more complicated, large system as functionalities were expanded on. The class diagram that reflects the parking system is displayed in Appendix B, and each association, dependency, and composition are noted. When the project began, the class diagram was significantly smaller, as outlined in Figure 1.

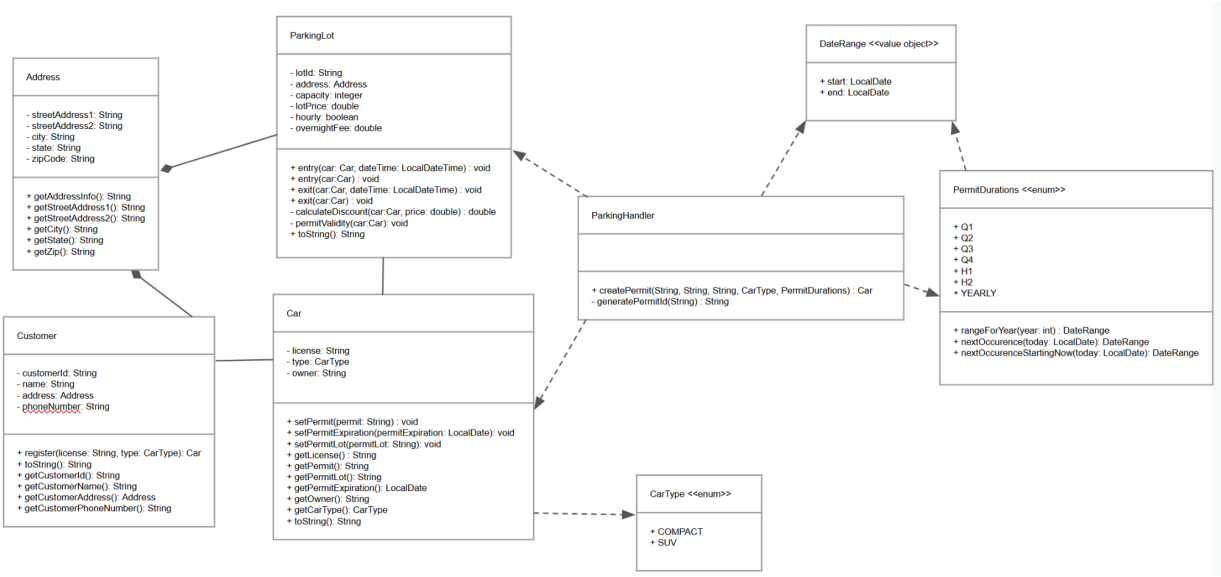


Figure 1. First attempt at Class Diagram

The start of the project was only to comprise the start of the system's core components. Setting up the values for customer, parking lot, and car classes built the foundation for the parking office service to utilize in future efforts. Figure 2 outlines how the class diagram evolved after implementing charges and permit registrations.

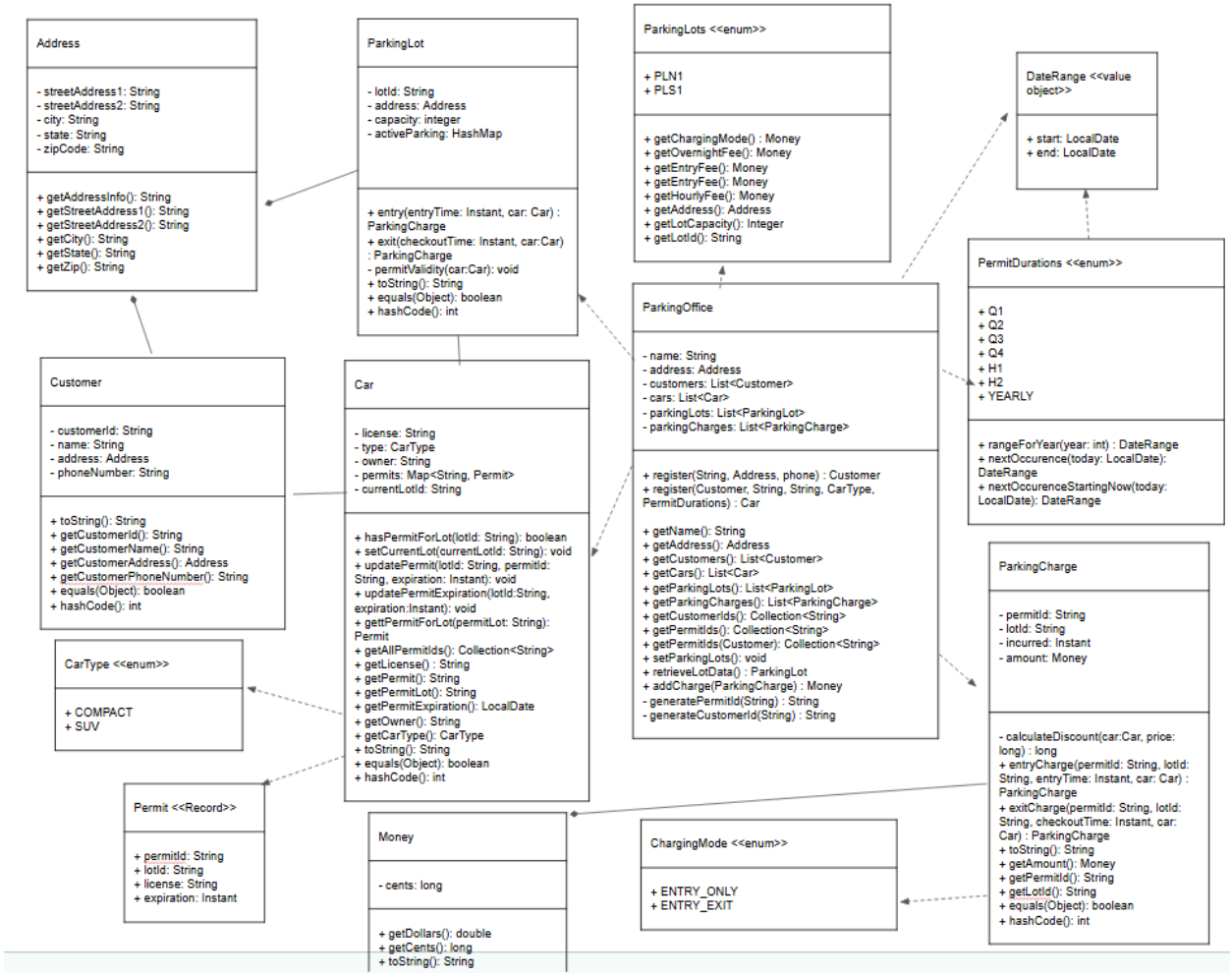


Figure 2. Class Diagram Expansion as Registration and Charges were added

Something that presented a challenge was trying to maintain a scalable system while developing further expansions to the parking classes. More complex systems came into play where the service, ParkingOffice, utilized the foundations set in earlier development stages, which then evolved into the final work of splitting up the service into more core components to register and charge customers with their cars. The final class diagram that is provided in Appendix B is the most complex of all previously developed diagrams, visually conceptualizing the growth of the system as a whole when even compared to the very first diagram. The

definition of the arrows utilized in the final class diagram are provided in Figure 3. To avoid redundancy, each relationship in the diagram won't be expanded on.

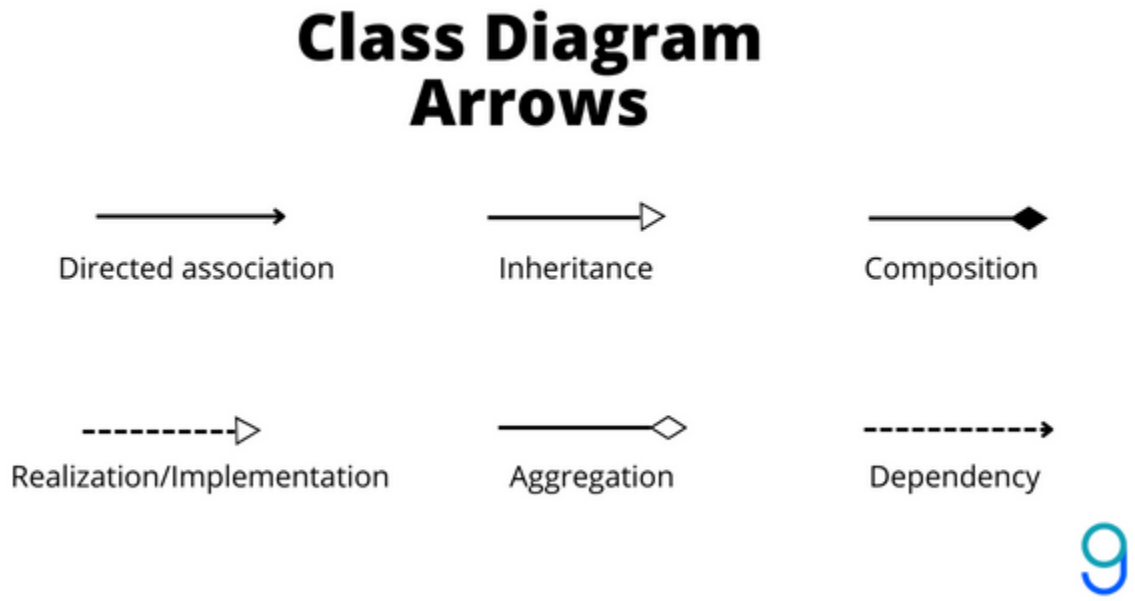


Figure 3. Class Diagram Arrows [Gleek. 2025.]

In summary, the final class diagram has many interconnected classes, specifically through aggregation, composition, dependency, and association. Aggregation is the relationship where the objects are loosely associated with each other, whereas composition is the direct association that one does not exist without the other. Dependency implies exactly as one may assume, which is that one class is dependent on another based on the direction of the arrow. Association differs from aggregation where it is *directly* associated to communicate between each other, but without the ownership semantics found in aggregation. (Gleek. 2025).

The Experience

The overall experience of this project was rewarding towards the end. Something I had struggled with throughout the development of this project was to not do too much lean-ahead work to ensure the overall structure of my code still matched the expectations of the course. Some mistakes were made where certain classes were developed before it was required, so when it came to implementing the new portions of the assignment, certain areas required refactoring. Although, many of the new implementations were suggestions, and my existing code had satisfied the functionality, therefore I didn't have to perform too much refactoring.

To create the class diagrams, I struggled for a while to find the best free option for developing UML Diagrams. For a bit, I had utilized Google Slides, as I had in my past experience messing with diagrams, although it became unorganized and complex. After some research, I found an online tool called LucidChart. This, though limited in the free features, provided UML Diagram components that kept my diagrams organized and simple.

Something else to mention was my experience when developing. I have previous experience developing in Java and IntelliJ, which assisted me with this project. Although, I could see this being exceedingly difficult for someone who has no previous experience. Setting up the environment in the starting stages of the project was difficult, as well as weighing benefits of utilizing streams versus loops, and certain language semantics.

Although there were some challenges, I foresee myself expanding on this project further even once the course is complete. There is a significant amount of redundancy I have in my

code due to the lean-ahead issues I ran into, so I plan to go back and refactor most of the code. Simplifying and increasing efficiency of code is always worth it, especially when there are multiple layers of accessors. Something I believe could've possibly made this project easier was the integration of a basic database system. I had integrated enumerations to keep track of parking lots, cars, etc., and I feel that a basic database (particularly a simple NoSQL Database) would have benefitted me. Instead of having to handle Java complications, I could consistently keep track of customers, associated cars, and parking lots in a simpler way. This is something I'm planning to implement to simplify more of the system in the future.

Conclusion

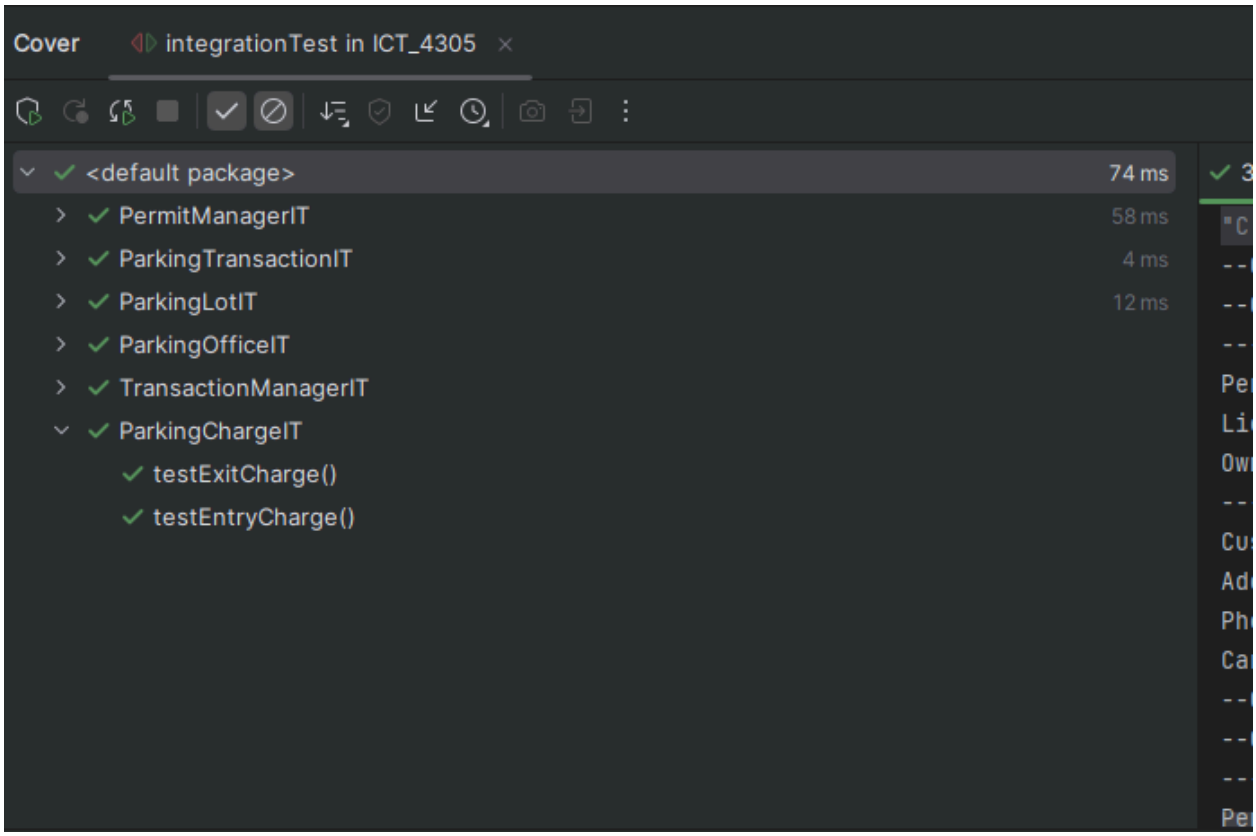
Though there were frustrations and challenges, the overall system satisfies the basic functionality requirements for the parking system. The system handles registrations for both customers and cars for permits, handles different types of charging based on parking lots the university owns, and stores charges based on each customer and relevant metadata. Certain improvements, such as the implementation of a NoSQL Database, will soon be implemented on this project to have better data storage and increase efficiency of the overall system.

References

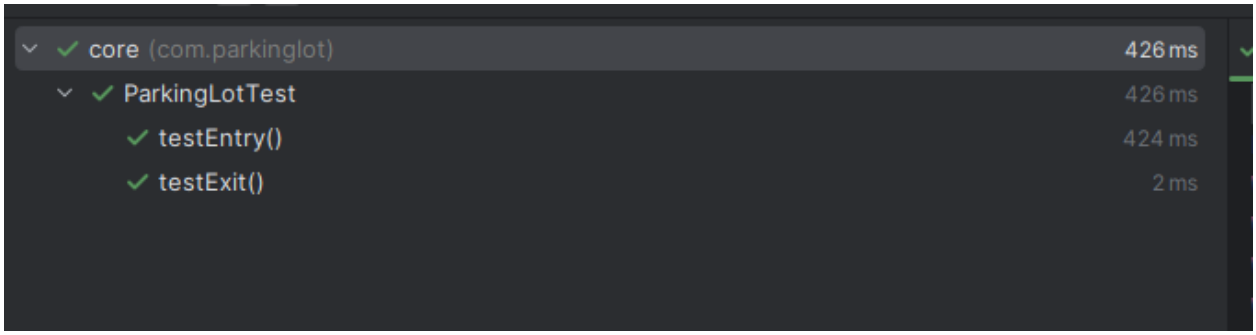
Gleek. 2025. “Diagram Maker for Developers.” Gleek. Accessed November 14. https://www.gleek.io/blog/class-diagram-arrows.

Appendix A: Test Execution, Compilation and Coverage

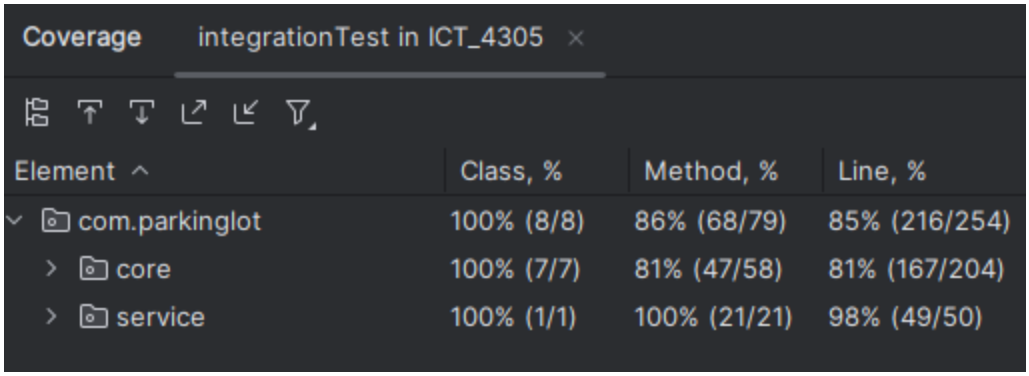
All Integration Tests Passing



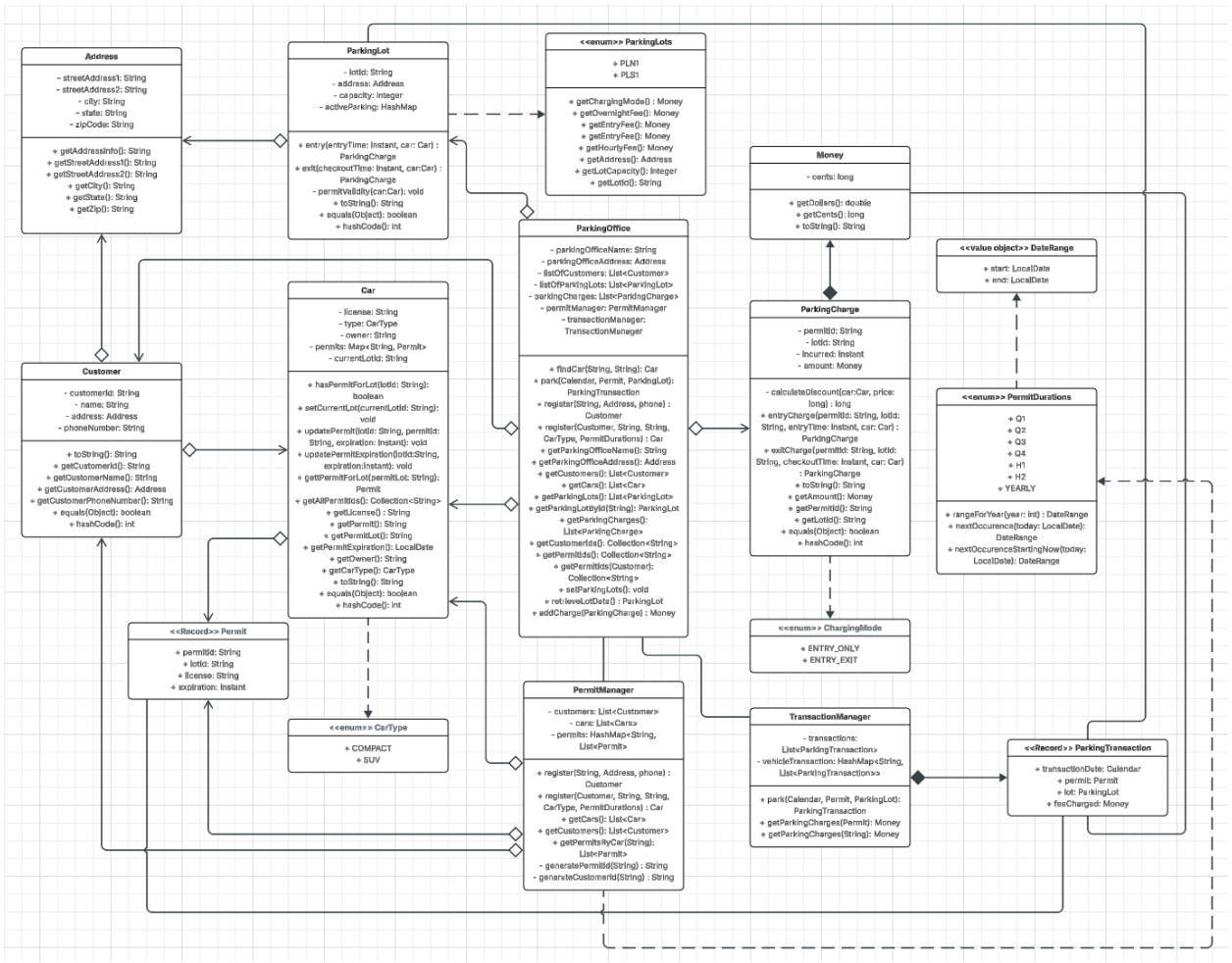
All Unit Tests Passing



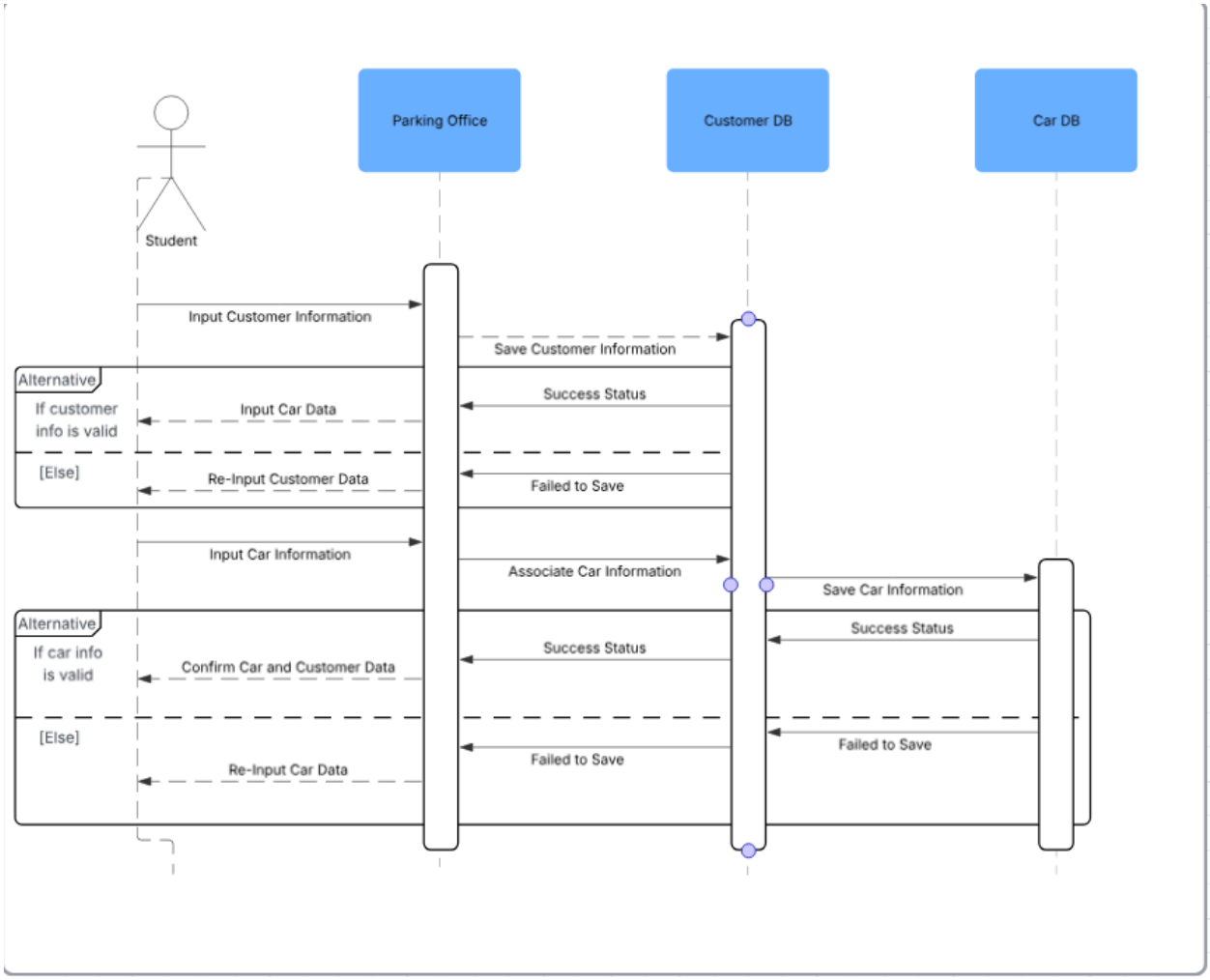
Coverage Numbers


Coverage
integrationTest in ICT_4305
Element
Class, %
Method, %
Line, %
com.parkinglot
100% (8/8)
86% (68/79)
85% (216/254)
core
100% (7/7)
81% (47/58)
81% (167/204)
service
100% (1/1)
100% (21/21)
98% (49/50)

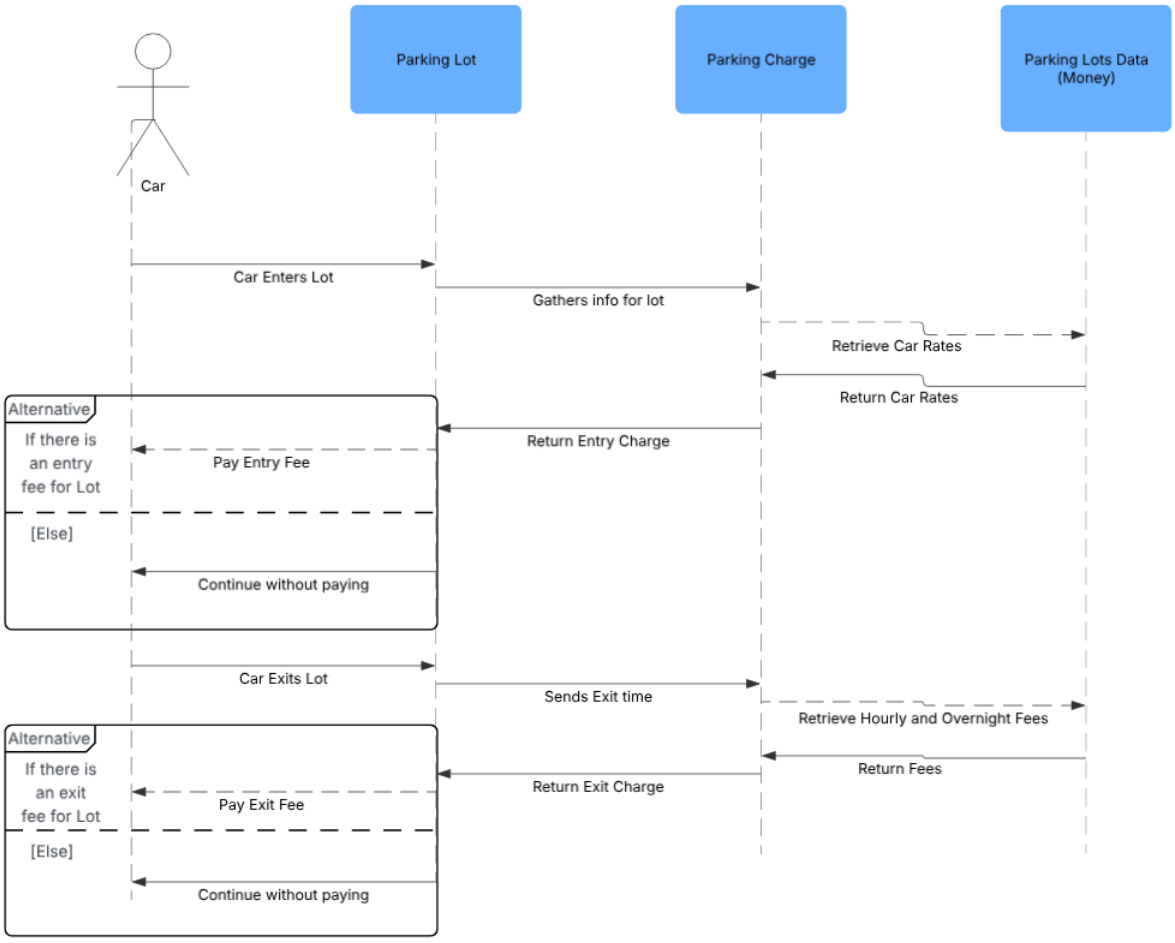
Appendix B: Updated Class Diagram



Appendix C: Sequence Diagram - Customer registering a Car and receiving a Parking Permit



Appendix D: Sequence Diagram - Car entering a Parking Lot, resulting in the Customer incurring a charge



Appendix E: Interaction Diagram

